

RECURSIVIDADE

LINGUAGEM DE PROGRAMAÇÃO

TALITA BERBEL

talita.berbel@fatec.sp.gov.br

Fatec
Sorocaba
José Crespo Gonzales

CP
Centro
Paula Souza

AGENDA

- Introdução
- Definição
- Execução
- Usabilidade
- Cálculo de fatorial
- Função recursiva
- Conclusões
- Exercícios
- Dúvidas



INTRODUÇÃO

- Pensando sobre:



DEFINIÇÃO

- Uma função que chama a si mesma direta ou indiretamente, é chamada de **recursiva**.
- Recursividade é um dos pilares da Teoria da Computação e do pensamento matemático, e é suportada por praticamente todas as linguagens de programação importantes.



DEFINIÇÃO

- Possibilidades de recursão:
 - **Recursão direta:** a função A chama a própria função A.
 - **Recursão indireta:** a função A chama uma função B que, por sua vez, chama A.
- Nenhum programa e nem função pode ser exclusivamente definido por si.
 - Um programa seria um loop infinito.
 - Uma função teria definição circular.
 - **Condição de parada:** Permite que o procedimento pare de se executar.

EXECUÇÃO

- Quando uma chamada de função é feita, é criado um registro de ativação na **pilha de execução** do programa.
- O registro de ativação guarda:
 - Os parâmetros e variáveis locais da função e
 - O ponto de retorno da função.
- Quando a função termina, o registro de ativação é desempilhado e a execução volta ao subprograma que chamou a função.

USABILIDADE

- Recursividade é válida para **algoritmos complexos**, cuja a implementação iterativa é complexa e normalmente requer o uso explícito de uma pilha.
- Exemplos:
 - Definições matemáticas: fatorial, Fibonacci
 - Caminhamento em Árvores (pesquisa, *backtracking*)
 - Algoritmos de divisão e conquista (Ex. Quicksort)
 - Processamento de expressões



CÁLCULO DE FATORIAL

- Exemplo clássico: calcular o fatorial de um número dado.
- Relação de recorrência da **forma iterativa**

$$n! = n \times (n - 1) \times \cdots \times 2 \times 1 = \prod_{k=1}^n k$$

```
long int fatorial(long int num)
{
    int i;
    long int fat=1;
    for(i=1;i<=num;i++)
        fat*=i;
    return fat;
}
```

- Relação de recorrência da **forma recursiva**

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ (n - 1)! \times n & \text{if } n > 0 \end{cases}$$

← Caso base ou condição de parada

← Chamada recursiva

CÁLCULO DE FATORIAL



- Exemplo do fatorial do número 3 usando a **recursão**:

$$3! = 3 * 2 * 1$$

ou

$$n! = n * (n-1)!$$

$$3! = 3 * 2!$$

$$2! = 2 * 1!$$

$$1! = 1 * 0!$$

$$0! = 1$$

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ (n-1)! \times n & \text{if } n > 0 \end{cases}$$

CÁLCULO DE FATORIAL

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ (n-1)! \times n & \text{if } n > 0 \end{cases}$$

- Exemplo do fatorial do número 3 usando a **recursão**:



FUNÇÃO RECURSIVA

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ (n-1)! \times n & \text{if } n > 0 \end{cases}$$

- **Protótipo:** `long int fatorial (long int num);`

- **Chamada da função dentro do programa:**

```
fat=fatorial(n);    ou    printf("%i", fatorial(n));
```

- **Definição:**

```
long int fatorial (long int num)
{
    if (num == 0)                //Condição de parada
        return 1;                //0! = 1
    return (num*fatorial(num-1)); //Chamada recursiva
}                                //3! = 3*(3-1)!
```

TIPOS DE DADOS

- Tipo dados numéricos primitivos **inteiros** em C (32 bits)

<i>Tipo</i>	<i>Bytes</i>	<i>Mín</i>	<i>Máx</i>	<i>Obs</i>
<code>char</code>	1	-128	127	Tipo inteiro de 1 byte com sinal
<code>unsigned char</code>	1	0	255	Tipo inteiro de 1 byte sem sinal
<code>short int</code>	2	-32.768	32.767	Tipo inteiro de 2 bytes com sinal
<code>unsigned short int</code>	2	0	65.535	Tipo inteiro de 2 bytes sem sinal
<code>long int</code>	4	-2.147.483.648	2.147.483.647	Tipo inteiro de 4 bytes com sinal
<code>unsigned long int</code>	4	0	4.294.967.295	Tipo inteiro de 4 bytes sem sinal
<code>long long int</code>	8	-9.223.372.036. 854.775.808	9.223.372.036. 854.775.807	Tipo inteiro de 8 bytes com sinal
<code>unsigned long long int</code>	8	0	18.446.744.073 .709.551.615	Tipo inteiro de 8 bytes sem sinal

Tipos com *unsigned* (sem sinal) utilizam todos os seus bytes para compor o seu valor

```

1  #include <stdio.h>
2
3  long int fatorial (long int num);
4
5  int main()
6  {
7      long int n, fat;
8
9      printf("Digite um numero: ");
10     scanf("%li",&n);
11
12     fat=fatorial(n);
13     printf("\nFatorial: %i",fat);
14 }
15
16 long int fatorial (long int num)
17 {
18     int i;
19     long int fat=1;
20     for(i=1;i<=num;i++)
21         fat*=i;
22     return fat;

```

Fatorial Iterativo

```

1  #include <stdio.h>
2
3  long int fatorial (long int num);
4
5  int main()
6  {
7      long int n, fat;
8
9      printf("Digite um numero: ");
10     scanf("%li",&n);
11
12     fat=fatorial(n);
13     printf("\nFatorial: %i",fat);
14 }
15
16 long int fatorial (long int num)
17 {
18     if (num == 0)
19         return 1;
20     return (num*fatorial(num-1));
21 }
22

```

Fatorial Recursivo

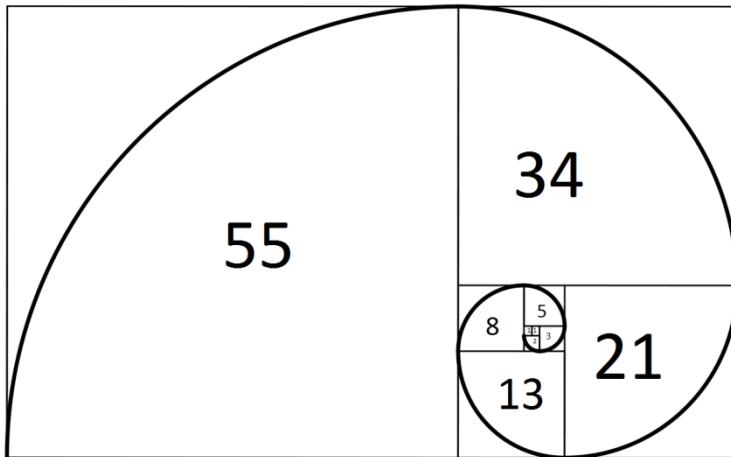
Chamada recursiva

CONCLUSÕES

- As rotinas recursivas tem uma versão iterativa equivalente;
- Linguagens, como as funcionais, que não possuem estrutura de repetição, a recursividade é utilizada para preencher essa falta;
- Linguagens imperativas tem a recursão como uma opção
 - Soluções recursivas para problemas recursivos são simples e elegantes, e permitem que o código evolua de forma natural;
 - Soluções iterativas para problemas recursivos costumam ser extensos e trabalhosos, criando dificuldade para adaptar o códigos para novas funcionalidade.

EXERCÍCIOS

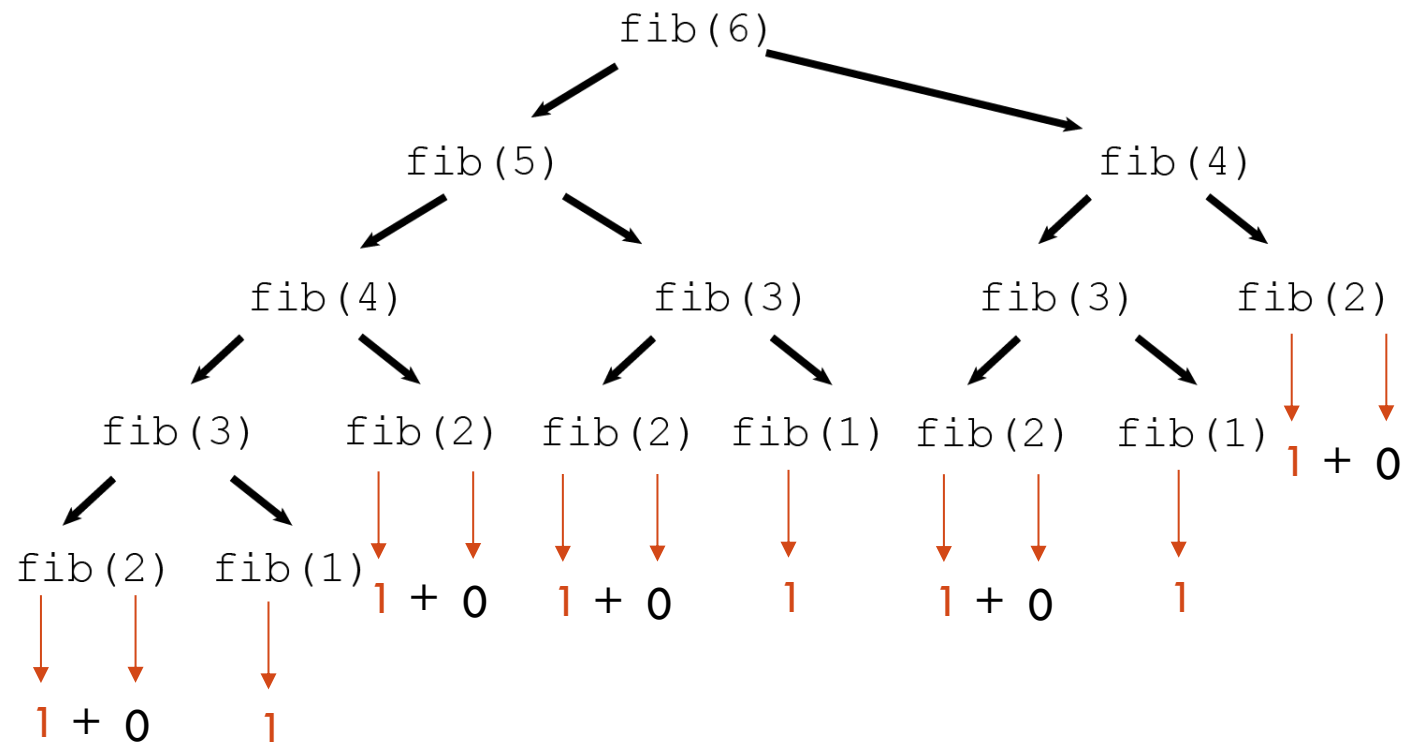
- A série de **Fibonacci** é formada pela sequência: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ... Desenvolver uma estrutura modular com uma função **recursiva** que recebe através de parâmetro um número indicando a posição do termo e retorna o valor correspondente na sequência de Fibonacci.



$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

EXERCÍCIO

- Exemplo de execução de Fibonacci do número 6:



$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

Se $F(2) = F(2-1) + F(2-2)$
 $= F(1) + F(0)$
 $= 1 + 0$

```

2  int fib(int n);
3  void mostrar_fib(int n);
4  int main()
5  {
6      int num;
7
8      printf("Informe o indice do termo na sequencia de Fibonacci desejada:\n");
9      scanf("%i", &num);
10
11     printf("\nO valor do indice %i de Fibonacci \x82 %i.", num, fib(num));
12     printf("\n\nSequencia: ");
13     mostrar_fib(num);
14     return 0;
15 }
16 int fib(int n)
17 {
18     if ((n == 0) || (n == 1))
19     {
20         return n;
21     }
22     else
23     {
24         return (fib(n-1) + fib(n-2));
25     }
26 }
27 void mostrar_fib(int n)
28 {
29     int i;
30     for(i=0;i<=n;i++)
31     {
32         printf("\nIndice[%i]= %i", i, fib(i));
33     }
34 }

```

$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

Essa é muito usada em programação, pois combina com índices de vetores (que começam em 0).

0, 1, 1, 2, 3, 5, 8, 13...

Já alguns livros de Matemática usam $F(1)=1$ e $F(2)=1$.

```

2  int fib(int n);
3  void mostrar_fib(int n);
4  int main()
5  {
6      int num;
7
8      printf("Informe a posicao do termo na sequencia de Fibonacci desejada:\n");
9      scanf("%i", &num);
10
11     printf("\n0 %io. termo da sequencia de Fibonacci \x82 %i.", num, fib(num));
12
13
14     printf("\n\nSequencia: ");
15     mostrar_fib(num);
16 }
17
18 int fib(int n)
19 {
20     if ((n == 1) || (n == 2))
21     {
22         return (1);
23     }
24     else
25     {
26         return(fib(n-1) + fib(n-2));
27     }
28 }
29
30 void mostrar_fib(int n)
31 {
32     int i;
33     for(i=1;i<=n;i++)
34     {
35         printf("\n%i", fib(i));
36     }
37 }

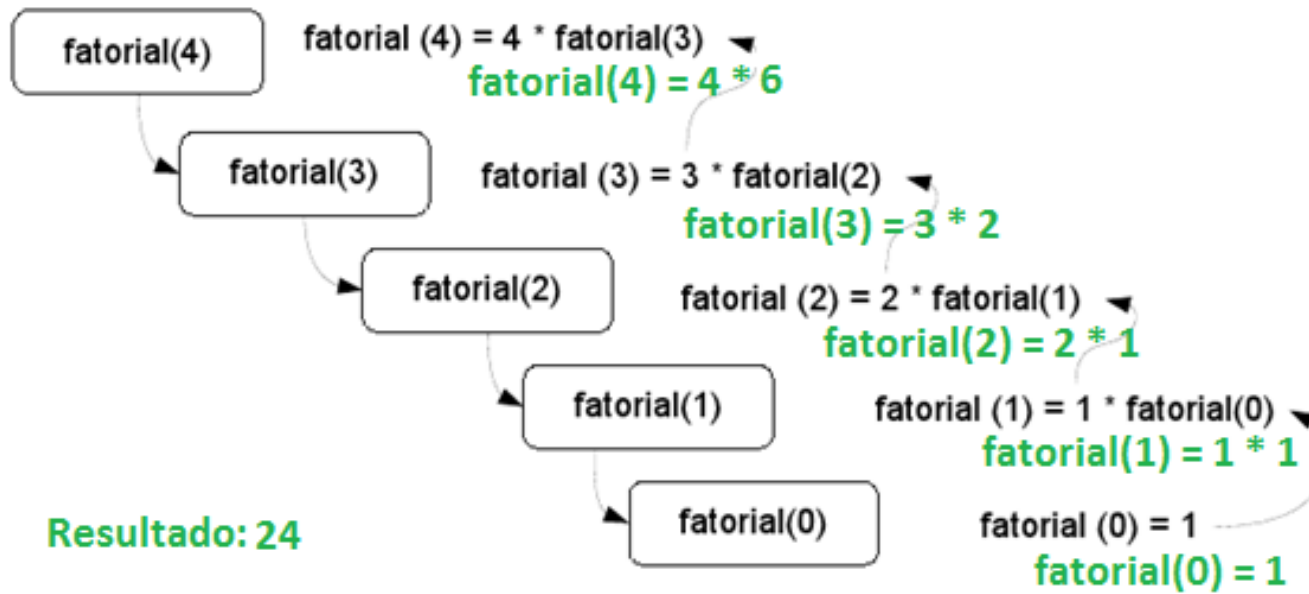
```

Alguns livros de Matemática usam $F(1)=1$ e $F(2)=1$.
1, 1, 2, 3, 5, 8, 13...

REPRESENTAÇÕES DE EXECUÇÃO

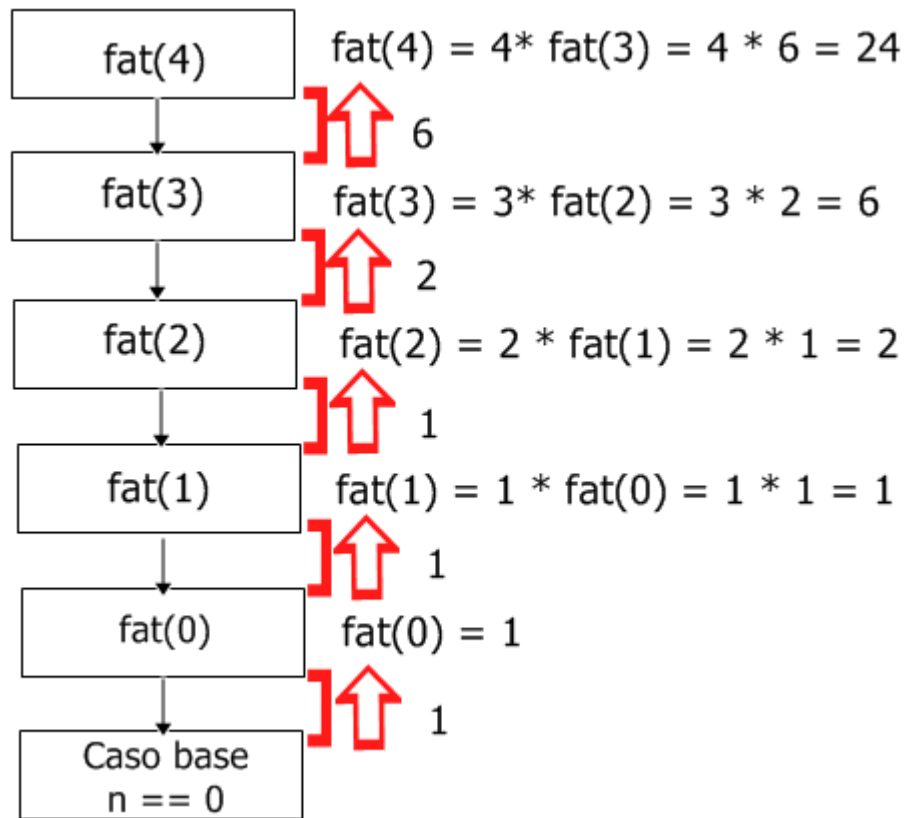
Fatorial recursivo

*Lembrem-se que a leitura é de baixo para cima!

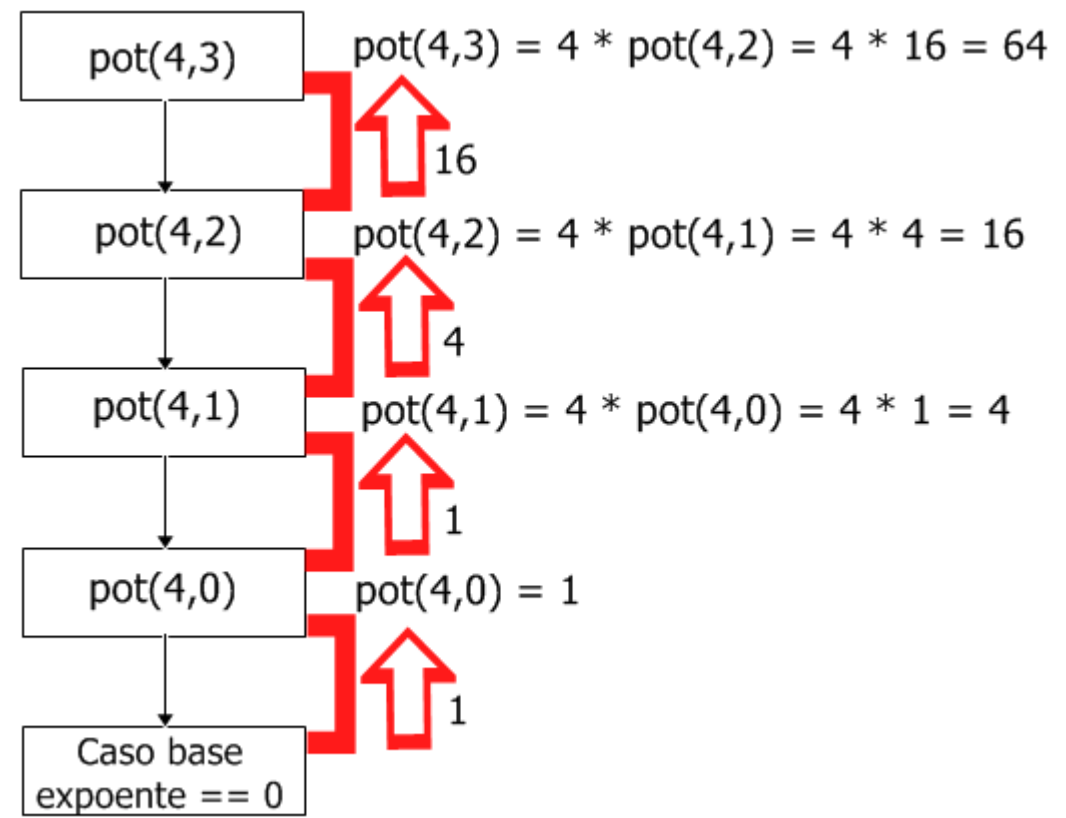


REPRESENTAÇÕES DE EXECUÇÃO

Fatorial recursivo



Potência Recursiva



RESOLUÇÃO

1) [Adaptação da questão 31 do Poscomp 2005] Considere o programa:

```
#include <stdio.h>
int m, n;

int FUN(int n)
{   int x;

    if (n < 1) return 1;
    else
    {   x = n * FUN(n - 1);
        m = m - 1;
        return m + x;
    }
}

int main(void)
{   scanf("%d %d", &m, &n);
    printf("%d %d %d\n", m, n, FUN(n));

    return 0;
}
```

Este programa, para os valores $m = 5$ e $n = 4$, tem como resultado:

- a) 5, 4, 120 b) 5, 4, 14400 c) 1, 4, 165 d) 5, 4, 5 e) 5, 4, 165

Considerando $m=5$ e $n=4$

FUN(n)

n	x	Operação
4	=4*....	
3	=3*....	
2	=2*....	
1	=1*....	

x=n*FUN(n-1)
m=m-1
return m+x

RESOLUÇÃO

1) [Adaptação da questão 31 do Poscomp 2005] Considere o programa:

```
#include <stdio.h>
int m, n;

int FUN(int n)
{   int x;

    if (n < 1) return 1;
    else
    {   x = n * FUN(n - 1);
        m = m - 1;
        return m + x;
    }
}

int main(void)
{   scanf("%d %d", &m, &n);
    printf("%d %d %d\n", m, n, FUN(n));

    return 0;
}
```

Este programa, para os valores $m = 5$ e $n = 4$, tem como resultado:

- a) 5, 4, 120 b) 5, 4, 14400 c) 1, 4, 165 d) 5, 4, 5 e) 5, 4, 165

Considerando $m=5$ e $n=4$

FUN(n)

n	x	Operação
4	=4*....	=4*41=164 m=2-1, volta 1+164
3	=3*....	=3*13=39, m=3-1, volta 2+39
2	=2*....	=2*5=10, m=4-1, volta 3+10
1	=1*....	=1*1 = 1, m=5-1, volta 4+1

$x = n * \text{FUN}(n-1)$, logo $1 * \text{FUN}(1-1) = 1$
 $m = m - 1$, logo $m = 4$
return $m + x = 4 + 1 = 5$

RESOLUÇÃO

1) [Adaptação da questão 31 do Poscomp 2005] Considere o programa:

```
#include <stdio.h>
int m, n;

int FUN(int n)
{ int x;

  if (n < 1) return 1;
  else
  { x = n * FUN(n - 1);
    m = m - 1;
    return m + x;
  }
}

int main(void)
{ scanf("%d %d", &m, &n);
  printf("%d %d %d\n", m, n, FUN(n));

  return 0;
}
```

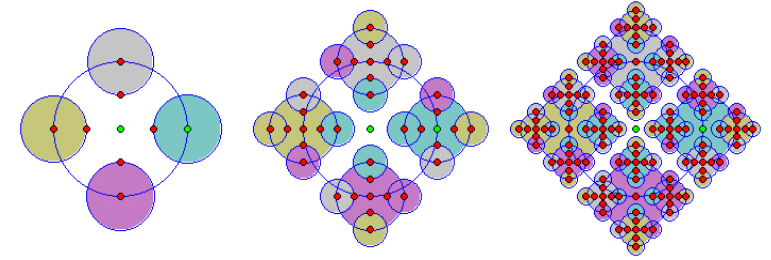
Este programa, para os valores $m = 5$ e $n = 4$, tem como resultado:

- a) 5, 4, 120 b) 5, 4, 14400 **c) 1, 4, 165** d) 5, 4, 5 e) 5, 4, 165

	n=4		m=1		FUN(n)
	n	x	Operação		
FUN(n)	4	=4*....	=4*41=164	m=2-1, volta 1+164 =	165
	3	=3*....	=3*13=39,	m=3-1, volta 2+39	
	2	=2*....	=2*5=10,	m=4-1, volta 3+10	
	1	=1*....	=1*1 = 1,	m=5-1, volta 4+1	

$x = n * FUN(n-1)$, logo $1 * FUN(1-1) = 1$
 $m = m - 1$, logo $m = 4$
return $m + x = 4 + 1 = 5$

RESOLUÇÃO

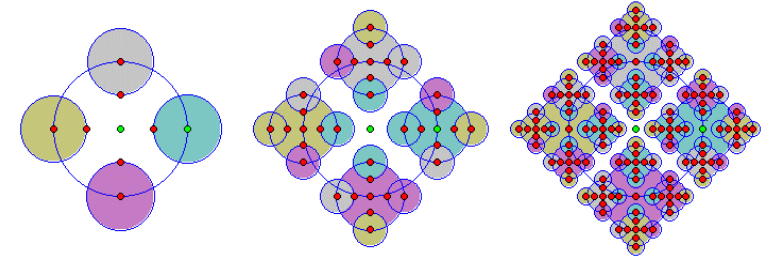


- 6) [Questão 21 do ENADE 2005, para Bacharel em Sistemas de Informação] No modo recursivo de representação, a descrição de um conceito faz referência ao próprio conceito. Julgue os itens abaixo, com relação à recursividade como paradigma de programação.
- I São elementos fundamentais de uma definição recursiva: o caso-base (base da recursão) e a reaplicação da definição.
 - II O uso da recursão não é possível em linguagens com estruturas para orientação a objetos.
 - III As linguagens de programação funcionais têm, na recursão, seu principal elemento de repetição.
 - IV No que diz respeito ao poder computacional, as estruturas iterativas e recursivas são equivalentes.
 - V Estruturas iterativas e recursivas não podem ser misturadas em um mesmo programa.

Estão certos apenas os itens

- a) I e IV.
- b) II e III.
- c) I, III e IV.
- d) I, III e V.
- e) II, IV e V.

RESOLUÇÃO



- 6) [Questão 21 do ENADE 2005, para Bacharel em Sistemas de Informação] No modo recursivo de representação, a descrição de um conceito faz referência ao próprio conceito. Julgue os itens abaixo, com relação à recursividade como paradigma de programação.
- I São elementos fundamentais de uma definição recursiva: o caso-base (base da recursão) e a reaplicação da definição.
 - II O uso da recursão não é possível em linguagens com estruturas para orientação a objetos.
 - III As linguagens de programação funcionais têm, na recursão, seu principal elemento de repetição.
 - IV No que diz respeito ao poder computacional, as estruturas iterativas e recursivas são equivalentes.
 - V Estruturas iterativas e recursivas não podem ser misturadas em um mesmo programa.

Estão certos apenas os itens

- a) I e IV.
- b) II e III.
- c) I, III e IV.
- d) I, III e V.
- e) II, IV e V.

RESOLUÇÃO

7) [Adaptação da questão 48 do Poscomp 2011] Observe a função recursiva a seguir.

```
int Prova(int N)
{
    if (N == 0)
        return 0;
    else
        return N * 2 - 1 + Prova(N - 1);
}
```

Considerando-se que essa função sempre será chamada com variável N contendo inteiros positivos, o seu valor de retorno será:

- a) O fatorial do valor armazenado em N .
- b) O valor armazenado em N elevado ao quadrado.
- c) O somatório dos N primeiros números inteiros positivos.
- d) O somatório dos N primeiros números pares positivos.
- e) 2 elevado ao valor armazenado em N .

Considerando diferentes entradas

N	Operação
0	volta ...
1	
2	
3	
4	
5	

RESOLUÇÃO

7) [Adaptação da questão 48 do Poscomp 2011] Observe a função recursiva a seguir.

```
int Prova(int N)
{
    if (N == 0)
        return 0;
    else
        return N * 2 - 1 + Prova(N - 1);
}
```

Considerando-se que essa função sempre será chamada com variável N contendo inteiros positivos, o seu valor de retorno será:

- a) O fatorial do valor armazenado em N .
- b) O valor armazenado em N elevado ao quadrado.**
- c) O somatório dos N primeiros números inteiros positivos.
- d) O somatório dos N primeiros números pares positivos.
- e) 2 elevado ao valor armazenado em N .

Considerando diferentes entradas

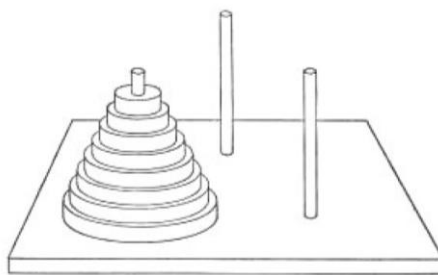
N	Operação
0	volta 0
1	volta $1*2 - 1 + \text{Prova}(0) = 1*2-1+0=1$
2	volta $2*2 - 1 + \text{Prova}(1) = 2*2-1+1=4$
3	volta $3*2 - 1 + \text{Prova}(2) = 3*2-1+4=9$
4	volta $4*2 - 1 + \text{Prova}(3) = 4*2-1+9=16$
5	volta $5*2 - 1 + \text{Prova}(4) = 5*2-1+16=25$

RESOLUÇÃO

8) [Questão 51 do ENADE 2005, para Ciência da Computação e Engenharia da Computação]

No famoso jogo da Torre de Hanoi, é dada uma torre com discos de raios diferentes, empilhados por tamanho decrescente em um dos três pinos dados, como ilustra a figura acima. O objetivo do jogo é transportar-se toda a torre para um dos outros pinos, de acordo com as seguintes regras: apenas um disco pode ser deslocado por vez, e, em todo instante, todos os discos precisam estar em um dos três pinos; além disso, em nenhum momento, um disco pode ser colocado sobre um disco de raio menor que o dele; é claro que o terceiro pino pode ser usado como local temporário para os discos. Imaginando que se tenha uma situação em que a torre inicial tenha um conjunto de 5 discos, qual o número mínimo de movimentações de discos que deverão ser realizadas para se atingir o objetivo do jogo?

- a) 25
- b) 28
- c) 31
- d) 34
- e) 38



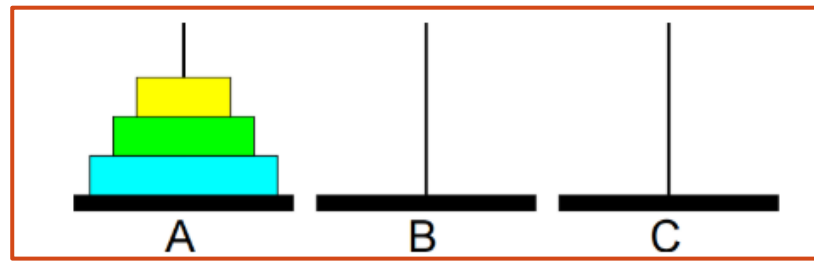
EXERCÍCIOS

Torres de Hanói

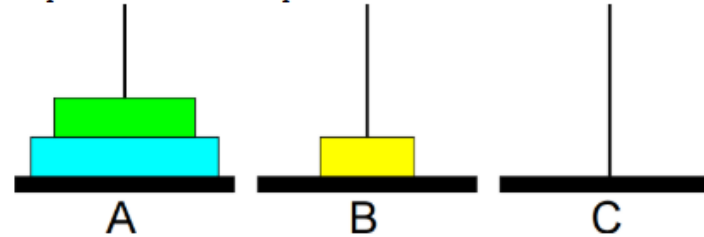
São dados um conjunto de n discos com diferentes tamanhos e três bases A, B e C.

O problema consiste em imprimir os passos necessários para transferir os discos da **base A** para a **base B**, usando a **base C** como auxiliar, nunca colocando discos maiores sobre menores.

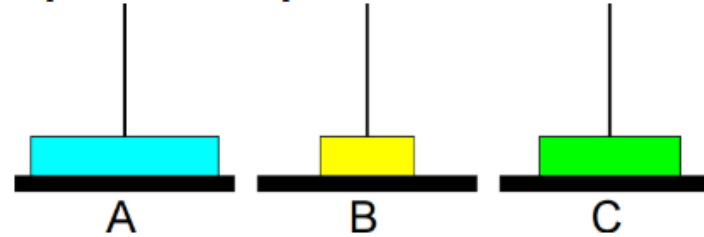
Inicial:



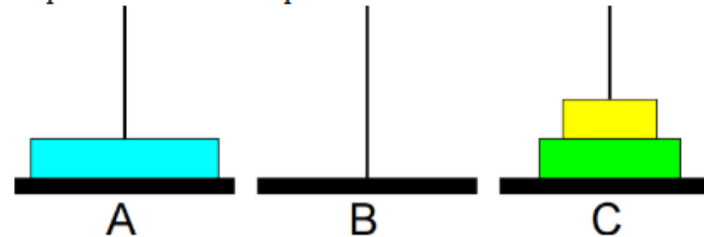
1º passo: Mover de A para B.



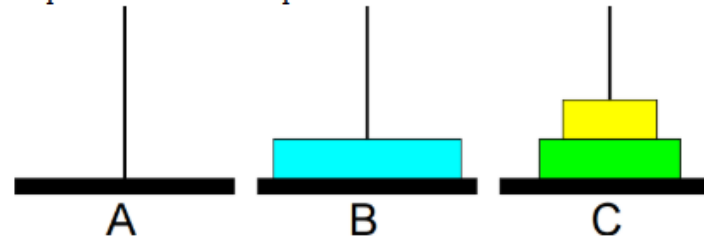
2º passo: Mover de A para C.



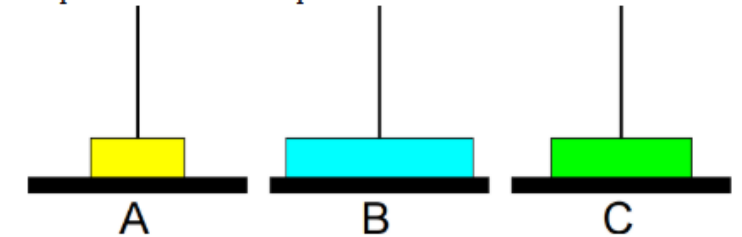
3º passo: Mover de B para C.



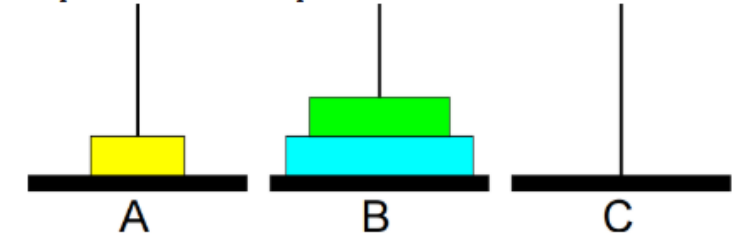
4º passo: Mover de A para B.



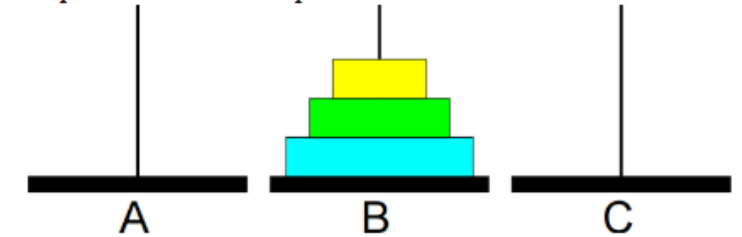
5º passo: Mover de C para A.



6º passo: Mover de C para B.

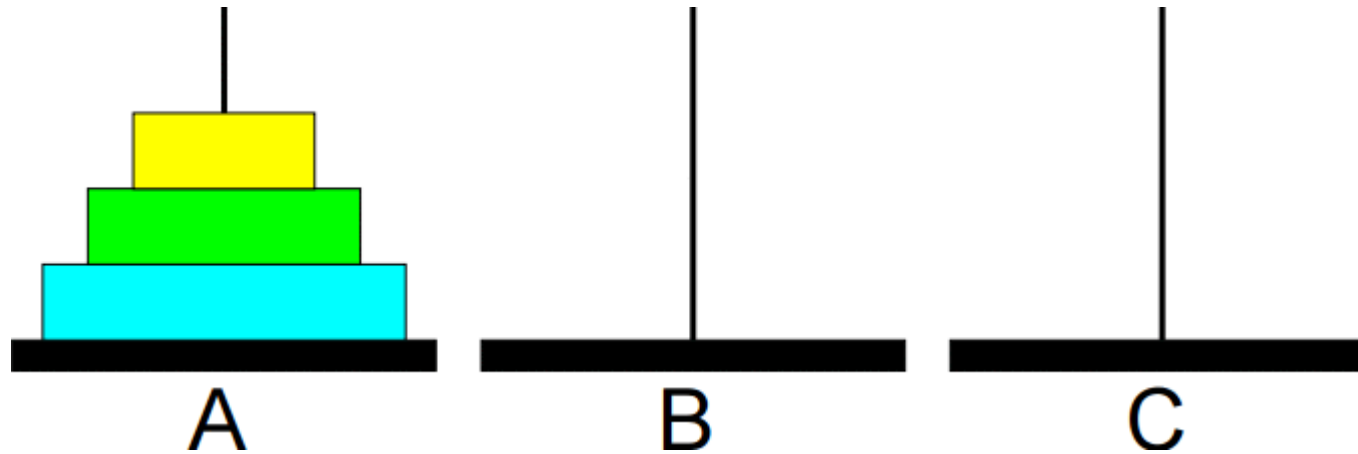


7º passo: Mover de A para B.



EXERCÍCIOS

- Torres de Hanói.



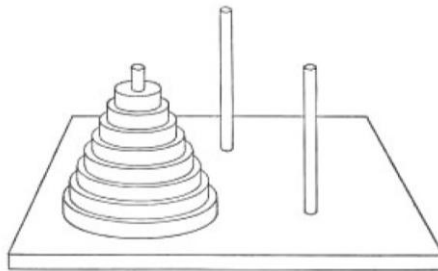
RESOLUÇÃO

<https://www.ime.usp.br/~leo/imatica/programas/hanoi/index.html>

8) [Questão 51 do ENADE 2005, para Ciência da Computação e Engenharia da Computação]

No famoso jogo da Torre de Hanoi, é dada uma torre com discos de raios diferentes, empilhados por tamanho decrescente em um dos três pinos dados, como ilustra a figura acima. O objetivo do jogo é transportar-se toda a torre para um dos outros pinos, de acordo com as seguintes regras: apenas um disco pode ser deslocado por vez, e, em todo instante, todos os discos precisam estar em um dos três pinos; além disso, em nenhum momento, um disco pode ser colocado sobre um disco de raio menor que o dele; é claro que o terceiro pino pode ser usado como local temporário para os discos. Imaginando que se tenha uma situação em que a torre inicial tenha um conjunto de 5 discos, qual o número mínimo de movimentações de discos que deverão ser realizadas para se atingir o objetivo do jogo?

- a) 25
- b) 28
- c) 31
- d) 34
- e) 38



Considerando números de discos diferentes

n	Número de movimentos
1	1
2	$1+1+1=3$
3	$3+1+3=7$
4	$7+1+7=15$
5	
6	
8	
10	
15	

RESOLUÇÃO

8) [Questão 51 do ENADE 2005, para Ciência da Computação e Engenharia da Computação]

No famoso jogo da Torre de Hanoi, é dada uma torre com discos de raios diferentes, empilhados por tamanho decrescente em um dos três pinos dados, como ilustra a figura acima. O objetivo do jogo é transportar-se toda a torre para um dos outros pinos, de acordo com as seguintes regras: apenas um disco pode ser deslocado por vez, e, em todo instante, todos os discos precisam estar em um dos três pinos; além disso, em nenhum momento, um disco pode ser colocado sobre um disco de raio menor que o dele; é claro que o terceiro pino pode ser usado como local temporário para os discos. Imaginando que se tenha uma situação em que a torre inicial tenha um conjunto de 5 discos, qual o número mínimo de movimentações de discos que deverão ser realizadas para se atingir o objetivo do jogo?

- a) 25
- b) 28
- c) 31
- d) 34
- e) 38

$$\begin{cases} T(n) = 2 \cdot T(n - 1) + 1 \\ T(1) = 1 \end{cases}$$

Considerando números de discos diferentes

n	Número de movimentos
1	$T(1)=1$
2	$T(2)=2 \cdot T(1)+1 = 3$
3	$T(3)=2 \cdot T(2)+1 = 7$
4	
5	
6	
8	
10	
15	

RESOLUÇÃO

8) [Questão 51 do ENADE 2005, para Ciência da Computação e Engenharia da Computação]

No famoso jogo da Torre de Hanoi, é dada uma torre com discos de raios diferentes, empilhados por tamanho decrescente em um dos três pinos dados, como ilustra a figura acima. O objetivo do jogo é transportar-se toda a torre para um dos outros pinos, de acordo com as seguintes regras: apenas um disco pode ser deslocado por vez, e, em todo instante, todos os discos precisam estar em um dos três pinos; além disso, em nenhum momento, um disco pode ser colocado sobre um disco de raio menor que o dele; é claro que o terceiro pino pode ser usado como local temporário para os discos. Imaginando que se tenha uma situação em que a torre inicial tenha um conjunto de 5 discos, qual o número mínimo de movimentações de discos que deverão ser realizadas para se atingir o objetivo do jogo?

- a) 25
- b) 28
- c) 31
- d) 34
- e) 38

$$\begin{cases} T(n) = 2 \cdot T(n - 1) + 1 \\ T(1) = 1 \end{cases}$$

Considerando números de discos diferentes

n	Número de movimentos
1	1
2	3
3	7
4	15
5	31
6	63
8	255
10	1 023
15	32 767

RESOLUÇÃO

8) [Questão 51 do ENADE 2005, para Ciência da Computação e Engenharia da Computação]

No famoso jogo da Torre de Hanoi, é dada uma torre com discos de raios diferentes, empilhados por tamanho decrescente em um dos três pinos dados, como ilustra a figura acima. O objetivo do jogo é transportar-se toda a torre para um dos outros pinos, de acordo com as seguintes regras: apenas um disco pode ser deslocado por vez, e, em todo instante, todos os discos precisam estar em um dos três pinos; além disso, em nenhum momento, um disco pode ser colocado sobre um disco de raio menor que o dele; é claro que o terceiro pino pode ser usado como local temporário para os discos. Imaginando que se tenha uma situação em que a torre inicial tenha um conjunto de 5 discos, qual o número mínimo de movimentações de discos que deverão ser realizadas para se atingir o objetivo do jogo?

- a) 25
- b) 28
- c) 31**
- d) 34
- e) 38

Fórmula Direta: $2^n - 1$

$$\begin{cases} T(n) = 2 \cdot T(n - 1) + 1 \\ T(1) = 1 \end{cases}$$

Considerando números de discos diferentes

n	Número de movimentos
1	1
2	3
3	7
4	15
5	31
6	63
8	255
10	1 023
15	32 767

RESOLUÇÃO

Fórmula Direta: $2^n - 1$

$$\begin{cases} T(n) = 2 \cdot T(n - 1) + 1 \\ T(1) = 1 \end{cases}$$

O algoritmo para resolver a quantidade de movimentos para mover os discos na Torres de Hanói:

moverHanoi(n, A, C, B) - mover n discos de A para C, usando a haste B

se $n == 1$, então mover disco do topo da haste A para a haste C - *final de recorrência*

senão

moverHanoi(n-1, A, B, C) - *mover otimamente n-1 discos de A para C (libera o ultimo)*

mover disco do topo da haste A para a haste C

moverHanoi(n-1, B, C, A) - *mover otimamente n-1 discos de B para C*

DÚVIDAS

- Questionamentos?

